

Secure Container Orchestration in the Cloud: Policies and Implementation*

Gabriel P. Fernandez
Technische Universität - Dresden
Dresden, Germany
gabriel.pereira_fernandez@tu-dresden.de

Andrey Brito
Universidade Federal de Campina Grande
Campina Grande, Brazil
andrey@computacao.ufcg.edu.br

ABSTRACT

In the late few years, cloud computing has been moving towards becoming the predominant infrastructure paradigm because of its scale economy advantages. Consequently, a great deal of sensitive and valuable information has begun to inhabit the cloud environment. At the current stage, a large amount of organizations is yet to step to the public cloud world because, primarily, the loss of infrastructure ownership gives away control of such information, and that is generally perceived as a severe security risk.

Currently available cloud security platforms fail to present a process to accomplish a secure and cloud compatible operation flow. In this work we propose a policy that enforces data security in the cloud. To implement the cloud based components of that policy we created SCO (Secure Container Orchestrator) a container orchestration engine that makes use of the most recent hardware-based trusted execution environment technologies for data protection, in our case, Intel SGX. SCO implements SGX friendly container auto-scaling schemes, load balancing routing and packs SGX compatibility features. These characteristics enable the deployment of trusted applications in alignment with standard cloud practices.

We also compare SCO to the state-of-the-practice in container orchestration (Kubernetes), demonstrate the inherent costs of employing this security enhanced solution, and present a methodology to support business decisions regarding its utilization.

CCS CONCEPTS

• **Security and privacy** → **Hardware security implementation**; • **Computer systems organization** → **Cloud computing**;

KEYWORDS

Cloud Computing, Container Orchestration, Trusted Execution Environment, Secure Load Balancing, Identity Authentication

*Produces the permission block, and copyright information

Most of the work has been carried out while Gabriel P. Fernandez was at the Universidade Federal de Campina Grande.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

SAC '19, April 8–12, 2019, Limassol, Cyprus

© 2019 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-5933-7/19/04...\$15.00

<https://doi.org/10.1145/3297280.3297296>

ACM Reference Format:

Gabriel P. Fernandez and Andrey Brito. 2019. Secure Container Orchestration in the Cloud: Policies and Implementation. In *The 34th ACM/SIGAPP Symposium on Applied Computing (SAC '19)*, April 8–12, 2019, Limassol, Cyprus. ACM, New York, NY, USA, Article 4, 8 pages. <https://doi.org/10.1145/3297280.3297296>

1 INTRODUCTION

Throughout the last decade, the mass migration of Internet-based applications towards cloud environment [12] has pressured cloud service providers to reinforce features related to multitenancy, for example, security and isolation. To that end, historically, standard hypervisor-level virtualization has been used.

Virtualization enables hosting a number of independent operating systems, linked to the underlying hardware through a hypervisor. Multitenant architectures have provided a higher application deployment granularity, thus greatly increasing cloud service providers mean resource utilization [14].

By the early 2010's, a new approach to virtualization emerged in industry. The organization of different application in containers, which, unlike the classical VM counterparts, ran on the same operating system kernel. Because each VM instantiates its own operating system, instantiation times for VMs are considerably high [10]. At times, specially in sudden high demand situations, the long start up time does not meet the commercial client needs.

Containers [20] achieve user isolation through the separation of user spaces, a feature available in Linux systems. This level of separation assures that different tenants have different process id spaces, network address spaces and file systems, while still sharing kernel functionality. A direct consequence of a shared kernel is that the instantiation for containers takes from milliseconds to seconds, while VM's can take up to minutes.

Although the deployment of applications is considerably improved by the usage of containers[21], also consequence of its good alignment with the DevOps philosophy [23], a major setback is that security is considerably looser than with VM's. From the VM perspective, invading its host machine is a task far more challenging than from a container. The shared infrastructure in a VM-hosted application is the hypervisor, a simple and somewhat monolithic attack surface which offers few vulnerabilities, no matter the level of control that the tenant acquires over its VM instance. Containers, on the other hand, share the operating system kernel [20], which is complex and has a much larger number of possible exploits.

As containers become the standard application isolation solution, the migration from private infrastructure to the public cloud approaches its limits [9]. The main reason for the maintenance of private infrastructure is the concern regarding security [11]. In a

cloud environment, there are no traditional means to ensure the safety of sensitive data and code. If a cloud service provider is viewed as a malicious actor, the threat model becomes severely more alarming. Such an adversary has full control over the system stack, from the hardware level to the applications and network. As a consequence, sensitive data theft can be performed by tampering with code or data, or even with cold boot attacks [13].

It is our vision in this paper that the lack of tools to guarantee data privacy in the cloud has been preventing many users to enjoy its advantages. We propose a mechanism to reliably verify the legitimacy of a service, as well as privacy of communication, all considering a powerful server side adversary. By the use of hardware-based software protection and a signature verification phase done by a trusted third party, we grant the user ways to corroborate the authenticity of the application, or to reject it when falsification is detected. This mechanism has its implementation made possible by the cloud orchestration platform here presented, SCO (Secure Container Orchestrator). SCO is a container orchestrator [22] based on Docker that automates the hardware based security measures we propose and attacks the problems with cloud orchestration of remotely attested applications.

This work differs from other Trusted Execution Environment based solutions because it focus on the “remoteness” aspect. This is done by providing a methodology to validate the integrity of a cloud application. The Third Party Authentication Entity (TPAE) and the SCO client carry out this task. It also focus on the deployment aspect. SCO makes it possible to easily deploy by solving typical container related conflicts with SGX. More importantly, SCO helps solving deployment and remoteness conflicts overlooked by other works by implementing a load balancing policy and a container life cycle control that addresses remote attestation issues.

From a Cloud Vendor perspective, offering services with these security features opens up a new market opportunity. Customers still attached to private IT infrastructures, dealing with its high costs, should be attracted to services providing confidentiality guarantees.

Our contributions in this work are as follows: A solution for users to verify a cloud based SGX application’s integrity. A container orchestration platform capable of easily deploying such applications and ensure they will function in the cloud. A business support decision model to evaluate the costs of this container orchestration platform.

2 USING INTEL SGX TO IMPROVE CONFIDENTIALITY IN THE CLOUD

The classic software security measures cannot completely safeguard applications deployed remotely in a cloud [1]. One of the few successful approaches to data protection with an adversary that has high privileges in the host machine is hardware-based security [4, 5, 8]. Intel, as well as other vendors [4, 8], has created hardware cryptography data protection extensions to their platforms. Those hardware solutions, SGX (Software Guard Extensions) in Intel’s case, greatly contribute to avert data theft [6].

The SGX user may submit their code to form a kernel and hardware shielded enclave and, once its creation process is completed, external accesses to the enclave are blocked by the kernel. The enclave data is hardware encrypted and decrypted when entering and

leaving the processor, rendering it unreadable to an adversary without a side channel approach. Since it is encrypted when running on main memory, not even sophisticated and expensive techniques, like cold boot attacks [13], are successful.

SGX also provides a remote attestation protocol, which consists of a modified Sigma protocol to facilitate a Diffie-Hellman key exchange between the client and the server [6] and the extension’s identity verification done by the IAS (Intel Attestation Service). The exchange results in a shared symmetrical key used for communication with the process running inside the enclave. This is a very convenient feature to private communication between a client and a server residing in a potentially hostile host environment, where attackers have the ability to capture data before it reaches its destination.

3 RELATED WORK

The main work addressing the issue of protecting containers from high-privileged unauthorized software is SCONE (Secure Container Environment) [2], a sophisticated toolset that offers supposedly low-overhead secure containers protected by SGX. SCONE makes use of its own compilation mechanism to ensure that the user’s application executes in an SGX enclave.

SCO differs from SCONE since it applies no extra overhead in the application (SCONE calls can be up to 1.2x normal speed). Most importantly, SCO’s SGX SDK based approach leaves to the developer the task of tailoring their application according to their needs, which does not involve the bug prone recompilation process that adds a large base of automatically generated code from SCONE.

On more traditional research directions, protection from privileged code in VM based environments has been a continuously revisited subject in the last decade. Overshadow [3], Virtual Ghost [7] and *SP³* [24] are examples of systems that apply software based memory protection from potentially malicious OS calls.

Other works with remarkable characteristics are Minibox [16], a sandbox isolation based framework that protects both unreliable applications and OS’s from each other and SEGO [15], which uses trusted verifiable metadata to assure the authenticity of services in hostile environments.

Platform modules to create hardware based secure environments like containers and VM’s can be found in the fairly recent literature, represented by Flicker [18], which supports application data sealing and other features, but falls short in performance for applications that require a high amount of transitions between trusted and untrusted modes. A natural answer to this problem is to enlarge the trusted area of the system, which is done so by TrustVisor [17] and CloudVisor [25] by including the hypervisor as a trusted entity.

Finally, as an example of the motivation for this work, the cold boot attack is detailed in works such as the one from Halderman et al. [13]. In that work, the authors conclude that DRAM might have to be considered an untrusted component of the system when it comes to the storage of sensitive information.

4 SECURITY POLICY

The security model assumed here has as its main starting point the classification of the hardware infrastructure owner as untrusted. Beyond that, which is covered in other works, we also classify the

application deployment as a procedure done completely remotely and therefore out of the application developers reach. During that process, the cloud based adversary should be able to carry out any changes to the application.

In this scenario, a malicious cloud provider can elude any attempts to perform remote audits by instantiating the original unaltered application for inspection, but serving the final users with an compromised version of it.

The sheer use of remote attestation combined with memory region protection, as proposed in the literature, can still be circumvented by virtualizing the environment and accessing the data from the underlying infrastructure, e.g. running the VM hypervisor over another operating system and reading the supposedly kernel level protected memory region from it.

Overall, it is necessary that a secure system in this context enforces the following guarantees:

- (1) **Denial of attempts to read sensitive data in the memory:** Unauthorized read attempts, originating from the operating system level or other processes, regardless of the reader user privilege level, must be rejected. Ideally, physical attacks such as cold boots, should also be prevented.
- (2) **Establishment of a reliable communication channel:** An attacker may not be able to capture unencrypted packets in the network, or encrypted packet with accessible keys.
- (3) **Identity attestation:** The cloud based system must be able to prove to a remote user its security capabilities.
- (4) **Application integrity authentication:** It must not be possible for the attacker to modify the original application without such modifications being detected by a user before valuable data is sent.

The sole usage of SGX can satisfy the first requirement since the SGX enclaves are kernel level protected and, thus, unauthorized reads to it are rejected. Furthermore, SGX enclaves are fully encrypted in memory and its data only decrypted when loaded to the processors registers.

The SGX attestation process verifies the identity of the environment, certified by Intel's attestation service. It establishes as well a symmetrical encryption key through its exchange protocol, which covers the second and third requirements.

Thus, the use of SGX by itself creates protections to the users' data that can only be trespassed if, during the deployment, the cloud service provider inserts malicious code into the enclave. An example can be seen in the Listings 1 and 2, where the cloud service provider introduces a malicious function that sends a secret outside of the enclave before it has been protected by some obfuscation method.

Listing 1: Unprotected application part. Thread calls enclave function

```
int main()
{
    char *ret = (char *) calloc(
        secret_char_n + 1, 1);
    std::thread legit_thread(
        hide_secret_and_publish_string, eid, *ret);
    legit_thread.join();
}
```

Listing 2: Accessing sensitive data within the enclave

```
#include "enclave_t.h"
#include <string.h>

char* my_char = "A_user's_secret!"

void hide_secret_and_publish_string(char *ret)
{
    // introducing malicious function call!
    sgx_get_secret_string( char *ret )
    //
    do_some_work_that_protects( my_char );
    memcpy( ret, my_char, strlen( my_char ) );
}
/* Malicious function! */
void sgx_get_secret_string( char *ret )
{
    memcpy( ret, my_char, strlen( my_char ) );
}
```

To avoid the falsification of the original application, we explore a functionality found in SGX. When the SGX remote attestation process finishes, an encrypted Quote containing metadata about the enclave is shared with the user. It includes identifiers from the SGX processor and from the enclave. Among them is a unique enclave hash based identifier, named as MRENCLAVE.

As the MRENCLAVE can be used to identify an enclave, we propose an additional authentication step done by the application user. In this step a public trusted party outside of the cloud environment, the Third Party Authentication Entity (TPAE), is used as integrity verification authority. The TPAE receives the MRENCLAVE from the application owner before the deployment in the untrusted cloud. When an user is performing a remote attestation on the enclave in the cloud, it queries the TPAE about the validity of the resulting MRENCLAVE. If it matches the one previously published by the developer for that application, then the cloud provider cannot have performed any tampering with the enclave code, since that would change the MRENCLAVE. In the case that the MRENCLAVE has changed, the user will then be able to choose not to proceed with the operation. Figure 1 illustrates the interaction scheme with the use of the TPAE.

The TPAE would play a role similar to Certifying Entities in the context of Digital Certificates used in the TLS authentication system.

5 SECURE CONTAINER ORCHESTRATOR

The policy from the previous section relies on the cloud service provider to be able correctly employ SGX powered compute instances in order to grant security guarantees. That although is a challenge on itself, since the regular container orchestration utilization and deployment flow does not take into account SGX based applications requirements.

To attend to those needs, we present the Secure Container Orchestrator (SCO). A container orchestrator that features the functionalities to host SGX based applications. Its main components can be seen in Figure 2.

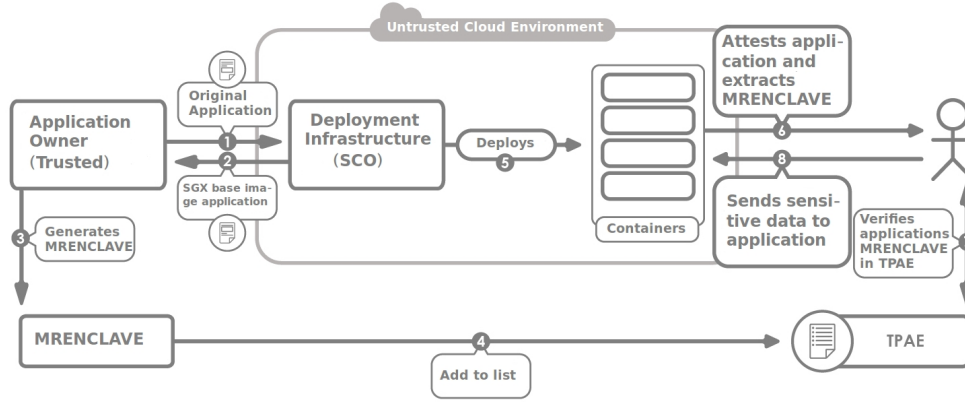


Figure 1: Untrusted environment hosted application verified with SGX and a TPAE utilization flow

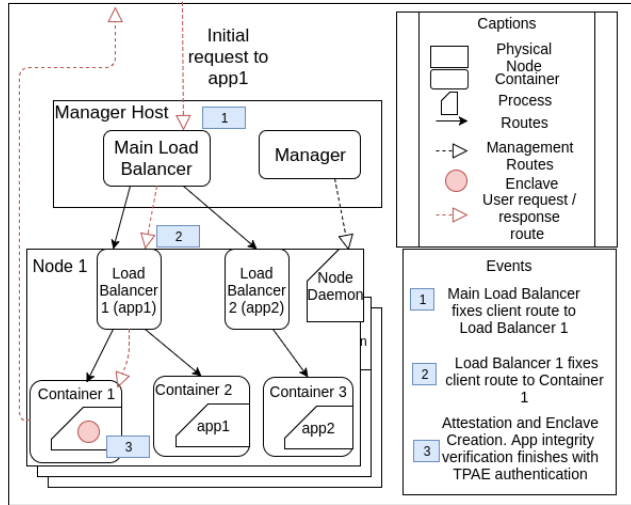


Figure 2: SCO Architecture – Components and Typical Utilization Flow.

Namely, the functionalities needed for SGX-based container orchestration and offered by SCO are the following:

- (1) **SGX virtual device accessible to the containers** Simply accessing the virtual device that allows for the container to utilize the SGX extension capabilities is not a feature readily available in the industries most popular container orchestration engines. The access to the SGX virtual device requires explicitly binding it from the host machine to the container. SCO natively implements SGX virtual device binding. Kubernetes, oppositely, does not allow for single device binding with its host machine. To access devices, containers must bind to all the host devices, which constitutes a severe security problem.
- (2) **Standard Docker images for SGX support** Docker uses a multilayer image system to facilitate the reuse of container

images. Taking advantage of this functionality, SCO uses a standard base image that possesses all the binaries, dependencies and environment tuning necessary to run SGX capable containers. This image is used as base upon which the user's Docker image is built. That measure greatly decreases the risk of incompatibilities, a task otherwise delegated to the user.

- (3) **IAS Remote attestation sensitive load balancing** The IAS remote attestation settles a connection-like relationship between a client and a specific container and so load balancing schemes must be able to establish a fixed route between a client's IP address and its attested container.

Ordinary session persistence will not suffice to bind a client to its attested container. That happens because HTTP-based session persistence requires HTTP-cookies to be enabled, something impossible in the purely TCP connection used by the IAS remote attestation process. Another approach would be to establish a correspondence between client IP addresses and containers by a hash table. That solution fails due to the elastic nature of the services. Whenever a new container is created or removed, a new table would have to be created, changing the client-container pairing.

To address this problem, SCO employs a modified multi-level load balancer system based on HAProxy¹, a free and popular load balancing alternative. The proposed scheme monitors generated Rsyslog log entries to search for remote attestation TCP packages. This package sequence is identifiable by our modified load balancer instances and serves to indicate that a client is carrying out a remote attestation and therefore will have an encrypted connection with the target container. When this connection is detected, our SCO Connection Daemon running in the container that hosts the HAProxy adds a rule binding the source to its destination container in the HAProxy configuration file until it is no longer active. If the load balancer is not an application load balancer, that is, it is an intermediate level load balancer, it will fix a route to some node that is running the application to which the request

¹ <http://www.haproxy.org/>

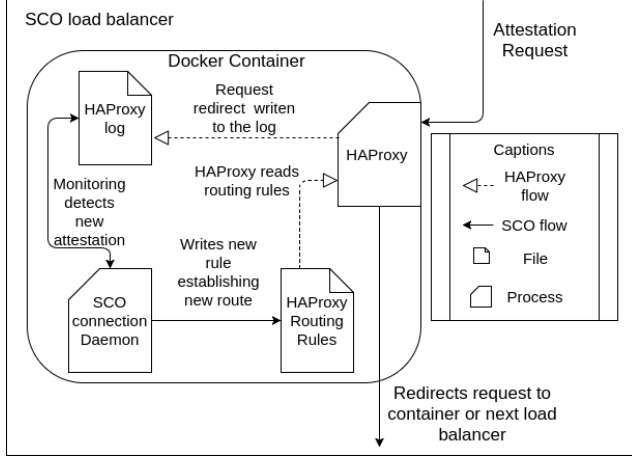


Figure 3: SCO Load Balancer Architecture.

was directed to. In Figure 2 the Main Load Balancer plays this role, establishing a fixed route to some application load balancer for app1. That is done throughout all load balancing levels, creating reliably fixed client-container routes. A scheme of SCO’s load balancers can be seen in Figure 3.

- (4) **Container removal control** Commercial container orchestration engines will either migrate or flush the application state to a storage volume to be picked up by another container when down scaling a service. However, data residing in the enclave is encrypted with a key stored in the SGX exclusive access registers that cannot be read by another processor or even another enclave. SCO undertakes this problem by postponing the removal of a container from a service that is at a low utilization level, but that still has active connections. To ratify that a container is no longer under utilization the following conditions are verified:
 - The maximum resend time of a TCP connection has been exceeded without any new packet being sent to the target container.
 - The container is not performing any I/O operations since the end of the time limit from the previous item.

This has the effect of nearly completely avoiding operations from being aborted due to container removal. In conventional container orchestration engines, containers will be removed despite still having active connections to clients under the assumption that applications are stateless. That assumption is inherently untrue in SGX applications. The remote attestation protocol and its key exchange phase, as detailed in Section 2, make all package exchanges with attested containers stateful on themselves.

It is noteworthy that not implementing this policy, in a long term scale down of a service, can even cause client starvation. That would happen in a scenario when a client continuously establishes new connections to containers in the service. One of these containers would start to effectuate transactions with a client. Due to the continuous scale down, there is a great likelihood that this container would be removed. The operation would have to be aborted and the process of connection establishment would be restarted. If the

scale down status was not revoked, this phenomenon would likely take place again, constituting starvation.

6 VALIDATION

In this section we compare SCO and Kubernetes, a market leading container orchestration engine.

6.1 SGX-based test application

The application used in the SCO validation had to meet the following requirements:

- SGX used to protect data in an untrusted environment;
- Be constituted of a scalable distributed architecture;
- Components easily isolated and replicated in containers.

So, we used an application named *Security and Privacy Preserving Data Aggregation* (SPPDA) found in the work of Silva et al. [19].

In SPPDA’s architecture, a set of smart meters produces consumption reads and these reads must be sent to the cloud service provider for billing, but must not be individually available in the untrusted environment. Let the set of meters be $S = \{SM_1, SM_2, \dots, SM_n\}$ and the set of measurements from meter i be e_{ij} , where time $j \in \{1, 2, \dots, t\}$.

Each smart meter attests an aggregation module running inside the enclave. A publish subscribe message bus is used for scalability. In this bus, an aggregator module a_{t_i} , such that $a_{t_i} \in \{a_{t_1}, a_{t_2}, \dots, a_{t_n}\}$ subscribes the topic t_i , in this case representing a region $t \in \{t_1, t_2, \dots, t_n\}$. The smart meters $SM_{t_i} | SM_{t_i} \in \{SM_{t_1}, SM_{t_2}, \dots, SM_{t_n}\}$, therefore, publish in the topic t_i and are aggregated by some a_{t_i} .

A non-SGX version of the application has been crafted for Kubernetes. The SGX Diffie-Hellmann key exchange protocol was replaced for a standard RSA/AES hybrid encryption scheme.

6.2 Experiment

We have chosen 3 performance benchmarks that, in our opinion, are the most important in order to preserve the cloud service provider’s efficiency. These are detailed below.

6.2.1 Auto Scaling. To stimulate the evaluated systems to perform scale up operations, we generate an increase in the number of smart meters publishing to a region’s topic, forcing the aggregator’s service mean CPU utilization level to surpass the threshold configured in the orchestrator.

Breaking this process in parts, being Δ_l the time necessary for the monitoring module to alert the management module and Δ_i , the container instantiation time, T is the total time.

We will also denote the ratio between the resulting time for SCO and for Kubernetes as α_1 , so:

$$T = \Delta_l + \Delta_i \quad (1)$$

$$T_{test} = \alpha_1 \times T_{control}, \quad (2)$$

6.2.2 Service Instantiation. We evaluate in this experiment the latency to fully instantiate a service in SCO and Kubernetes. We consider full instantiation when the last container of the minimum required on creation is executing.

Here, Δ_r is the time for the creation request to reach and be processed by the container orchestrator and Δ_i the time necessary to instantiate a container i such that $i \leq n$, the minimum amount of containers requested.

As in the previous experiment, the ratio between the times for the SCO and Kubernetes is α_2 :

$$T = \Delta_r + \max(\Delta_1, \Delta_2, \dots, \Delta_n) \quad (3)$$

$$T_{test} = \alpha_2 \times T_{control} \quad (4)$$

6.2.3 Application. Lastly, another factor that plays an important role in the overall cloud performance is the application being run itself. It is expected that the SGX-based application must present a significantly longer attestation time, compared to simple RSA key exchange.

The total time α_3 is simply the attestation time and can be expressed as:

$$T_{test} = \alpha_3 \times T_{control} \quad (5)$$

6.3 Business support methodology

Furthermore, we propose a metric to establish the total relative cost to use SCO in relation to Kubernetes. This total cost is comprised by the mean of the variables α_1 , α_2 and α_3 each of them being weighed by a factor w_i that represents how important the aspect ratio α_i is to the service. Thus different service profiles, set by the chosen w_i 's, should have different costs. Summarizing, this total cost, expressed by C_{total} , is:

$$C_{total} = \frac{(w_1\alpha_1 + w_2\alpha_2 + w_3\alpha_3)}{3} \quad (6)$$

To normalize the value of C_{total} , it is required that the w_1 , w_2 and w_3 are such that:

$$w_1 + w_2 + w_3 = 3 \quad (7)$$

6.4 Results

Figure 4 presents SCO and Kubernetes auto-scaling performance with up to 8 instances running. The linear behavior exhibited by both tools' performances helps validating the model proposed in the previous section that a fixed ratio explains well the performance difference between the tools.

The confidence intervals presented in this experiment are fairly well behaved, maintaining a safe distance between the curves.

In this scenario, we can use the means to generate an approximate α_1 . In Table 1 the means and each of their ratios μ_i such that $i \in \{2, 4, 6, 8\}$ can be seen.

Being α_1 the mean of all the μ_i , we reach a value of:

$$\alpha_1 = 2.37 \quad (8)$$

This value can be explained, mainly, by Kubernetes leverage on performing its calls directly onto the Docker daemon, while the SCO utilizes the host's Docker client through a call to the operating system. Auto-scaling is an operation fairly dominated by the container orchestration engine ability to quickly detect that some service utilization has surpassed or gone below some threshold. SCO relies on Docker client's slow container metadata probing and

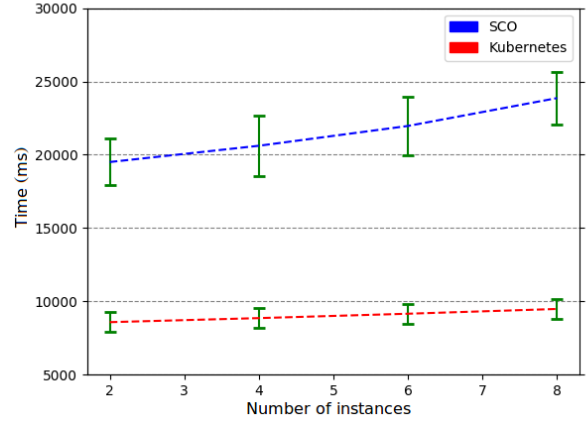


Figure 4: Auto-Scaling Latency with Confidence Intervals – SCO × Kubernetes

Table 1: Mean auto-scaling latencies (ms)

	SCO	Kubernetes	μ_i
2 instances	19627.12	8579.87	2.27
4 instances	20611.70	8856.25	2.32
6 instances	21963.37	9155.25	2.39
8 instances	24052.50	9480.25	2.53

then still needs to process the output text. The replacement of the current Docker API with a more efficient one that communicates with Docker daemon should significantly decrease the α_1 ratio and is left as future work.

This experiment also reveals a consistently high variance, as seen in Figure 4. That is as well due to the erratic performance of external operating system calls.

Figure 5 presents SCO's and Kubernetes' performances to instantiate a new service of up to 8 containers. Even clearer than in the previous experiment, linearity and small confidence intervals can be seen, with the confidence intervals ranging at least one degree of magnitude smaller than the means, which can be seen in Table 2.

Again, being α_2 the mean of all the μ_i , we reach a value of:

$$\alpha_2 = 5.67 \quad (9)$$

A sharp decrease in SCO's performance can be seen in this experiment, compared to the previous one, although the linearity is still preserved. We explain the loss in performance, to a great extent, with the instantiation of several enclaves, not performed by the conventional application run in Kubernetes. Although the calls for container creation are asynchronous, the enclave creation is not performed in parallel. This generates a queue of enclave creation calls that bottlenecks the performance. This is a very revealing result since this behavior should be observed whenever enclave application instances are deployed in sequence.

Kubernetes, in contrast, instantiates its containers completely in parallel. The conventional container application has a lower

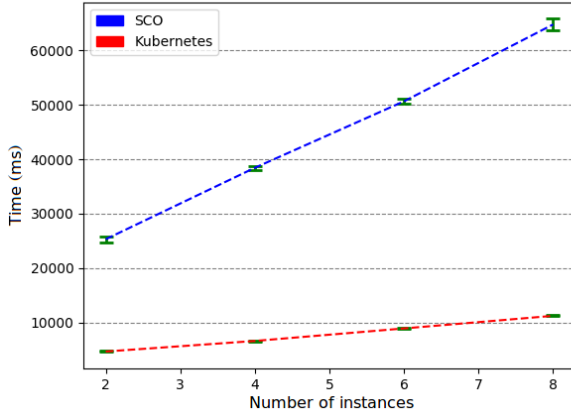


Figure 5: Service Instantiation Latency with Confidence Intervals – SCO × Kubernetes

Table 2: Service instantiation mean latency (ms)

	SCO	Kubernetes	μ_i
2 instances	25304.01	4664.12	5.42
4 instances	38435.62	6607.62	5.81
6 instances	50652.25	8908.62	5.68
8 instances	64776.75	11226.25	5.77

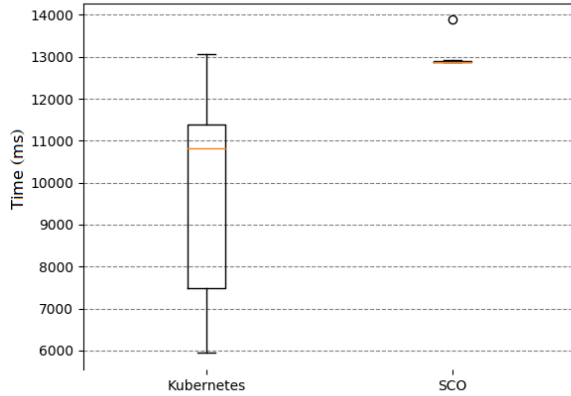


Figure 6: Control and test applications attestation latency

instantiation time in comparison with the SGX version run in SCO as well. Those factors should also take a heavy toll on α_2 .

A solution for the high ratio observed in α_2 is to prioritize instantiation in distinct physical nodes, thus increasing the level of parallelization.

Finally, the original SPPDA application, running on SCO, has been compared to its modified unsafe counterpart running on Kubernetes. The results can be seen in Figure 6.

Table 3: Application attestation statistics

	Mean	Conf. Interval (95%)	Std. Deviation
SCO	12977.45	[12775.65, 13179.25]	2432.78
Kubernetes	9722.27	[8008.13, 11436.41]	286.40

Possibly due to RSA’s prime factors choice, the unsafe Kubernetes version presented a much greater variance, but was consistently superior to the original application on SCO.

We model α_3 simply as the ratio between the means for original and modified latencies. These values and other relevant statistics can be seen in Table 3.

Taking these values in consideration, α_3 is the following:

$$\alpha_3 = 1.33 \quad (10)$$

7 CONCLUSION

In this paper we propose policies to assure secure deployment in an untrusted cloud, with Intel SGX in a protagonist role in meeting this requirement. This policy consists in the usage of the SGX’s MRENCLAVE and the participation of a Third Part Authentication Entity (TPAE) to verify the integrity of the running applications and services.

Next, to be able to execute native SGX applications in the cloud, we developed the Secure Container Orchestrator (SCO). SCO is a container orchestration engine designed to execute SGX applications by implementing dedicated container life cycle and load balance routing policies, and automatizing access to SGX-related resources like drivers and virtual devices.

Furthermore, we evaluate SCO’s performance in critical cloud operations, namely auto-scaling, service instantiation, and application attestation (in this case, not the orchestrator itself being evaluated, but rather a secure SGX application and its conventional version). This evaluation is done against a mainstream orchestration tool, Kubernetes, and the difference between them in each of these operations is summarized in the variables α_1 , α_2 and α_3 . These values are then used in a business decision support model that helps quantifying the costs of utilizing SCO as an orchestration solution in comparison with Kubernetes.

The experiments carried out in this study have hinted that SCO can be an interesting alternative for managing applications that use sensitive data in the cloud.

ACKNOWLEDGMENTS

This research was funded by the EU-BRA SecureCloud project (MCTI/RNP 3rd Coordinated Call)².

REFERENCES

- [1] Cloud Security Alliance. 2016. *The Treacherous 12 - Cloud Computing Top Threats in 2016*. Technical Report.
- [2] Gregor F. Arnaudov S., Trach B. 2016. SCONE: Secure Linux Containers with Intel SGX. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. USENIX Association.
- [3] Xiaoxin Chen, Tal Garfinkel, E. Christopher Lewis, Pratap Subrahmanyam, Carl A. Waldspurger, Dan Boneh, Jeffrey Dworkin, and Dan R.K. Ports. 2008. Overshadow:

²www.securecloudproject.eu

- A Virtualization-based Approach to Retrofitting Protection in Commodity Operating Systems. In *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XIII)*. ACM, New York, NY, USA, 2–13. <https://doi.org/10.1145/1346281.1346284>
- [4] ARM Corporation. 2016. ARM TrustZone. <https://developer.arm.com/technologies/trustzone>. [Online; accessed 01-May-2017].
- [5] Intel Corporation. 2016. Intel Software Guard Extensions (Intel SGX). Intel’s WebPage. <https://software.intel.com/en-us/sgx>.
- [6] Victor Costan and Srinivas Devadas. [n. d.]. *Intel SGX explained*. Technical Report. Cryptology ePrint Archive, Report 2016/086, 2016.
- [7] John Criswell, Nathan Dautenhahn, and Vikram Adve. 2014. Virtual Ghost: Protecting Applications from Hostile Operating Systems. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS ’14)*. ACM, New York, NY, USA, 81–96. <https://doi.org/10.1145/2541940.2541986>
- [8] Tom Woller David Kaplan, Jeremy Powell. 2013. AMD Memory Encryption White paper. http://amd-dev.wpengine.netdna-cdn.com/wordpress/media/2013/12/AMD_Memory_Encryption_Whitepaper_v7-Public.pdf. [Online; accessed 25-November-2016].
- [9] Cloud Endure. 2016. 2016 Cloud Migration Survey - Cloud Migration Challenges and Best Practices. Survey.
- [10] Wes Felten, Alexandre Ferreira, Ram Rajamony, and Juan Rubio. 2015. An updated performance comparison of virtual machines and Linux containers. *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)* (2015), 171–172. <https://doi.org/doi.ieeecomputersociety.org/10.1109/ISPASS.2015.7095802>
- [11] Diogo A. Fernandes, Liliana F. Soares, João V. Gomes, Mário M. Freire, and Pedro R. Inácio. 2014. Security Issues in Cloud Environments: A Survey. *Int. J. Inf. Secur.* 13 (2014), 113–170. <https://doi.org/10.1007/s10207-013-0208-7>
- [12] Ian T. Foster, Yong Zhao, Ioan Raicu, and Shiyong Lu. 2016. Cloud Computing and Grid Computing 360-Degree Compared. (2016).
- [13] J. Alex Halderman, Seth D. Schoen, Nadia Heninger, William Clarkson, William Paul, Joseph A. Calandrino, Ariel J. Feldman, Jacob Appelbaum, and Edward W. Felten. 2009. Lest We Remember: Cold-boot Attacks on Encryption Keys. *Commun. ACM* 52, 5 (May 2009), 91–98. <https://doi.org/10.1145/1506409.1506429>
- [14] IBM. 2016. 4 key design considerations for a multi-tenant cloud. IBM webpage. <https://www.ibm.com/blogs/cloud-computing/2016/08/design-considerations-multi-tenant-cloud/>.
- [15] Youngjin Kwon, Alan M. Dunn, Michael Z. Lee, Owen S. Hofmann, Yuanzhong Xu, and Emmett Witchel. 2016. Sego: Pervasive Trusted Metadata for Efficiently Verified Untrusted System Services. *SIGOPS Oper. Syst. Rev.* 50, 2 (March 2016), 277–290. <https://doi.org/10.1145/2954680.2872372>
- [16] Yanlin Li, Jonathan McCune, James Newsome, Adrian Perrig, Brandon Baker, and Will Drewry. 2014. MiniBox: A Two-Way Sandbox for x86 Native Code. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*. USENIX Association, Philadelphia, PA, 409–420. https://www.usenix.org/conference/atc14/technical-sessions/presentation/li_yanlin
- [17] Jonathan M. McCune, Yanlin Li, Ning Qu, Zongwei Zhou, Anupam Datta, Virgil Gligor, and Adrian Perrig. 2010. TrustVisor: Efficient TCB Reduction and Attestation. In *Proceedings of the 2010 IEEE Symposium on Security and Privacy (SP ’10)*. IEEE Computer Society, Washington, DC, USA, 143–158. <https://doi.org/10.1109/SP.2010.17>
- [18] Jonathan M. McCune, Bryan J. Parno, Adrian Perrig, Michael K. Reiter, and Hiroshi Isozaki. 2008. Flicker: An Execution Infrastructure for Tcb Minimization. *SIGOPS Oper. Syst. Rev.* 42, 4 (April 2008), 315–328. <https://doi.org/10.1145/1357010.1352625>
- [19] Leandro Ventura Silva, Rodolfo Marinho, Jose Luis Vivas, and Andrey Brito. 2017. Security and privacy preserving data aggregation in cloud computing. In *Proceedings of the Symposium on Applied Computing*. ACM, 1732–1738.
- [20] Stephen Soltesz, Herbert Pötl, Marc E. Fuczynski, Andy Bavier, and Larry Peterson. 2007. Container-based Operating System Virtualization: A Scalable, High-performance Alternative to Hypervisors. In *Proceedings of the 2Nd ACM SIGOPS/EuroSys European Conference on Computer Systems 2007 (EuroSys ’07)*. ACM, New York, NY, USA, 275–287. <https://doi.org/10.1145/1272996.1273025>
- [21] Andrea Tosatto, Pietro Ruiu, and Antonio Attanasio. 2015. Container-Based Orchestration in Cloud: State of the Art and Challenges. In *Ninth International Conference on Complex, Intelligent, and Software Intensive Systems, CISIS 2015, Santa Catarina, Brazil, July 8-10, 2015*. 70–75. <https://doi.org/10.1109/CISIS.2015.35>
- [22] Andrea Tosatto, Pietro Ruiu, and Antonio Attanasio. 2015. Container-Based Orchestration in Cloud: State of the Art and Challenges. In *Proceedings of the 2015 Ninth International Conference on Complex, Intelligent, and Software Intensive Systems (CISIS ’15)*. IEEE Computer Society, Washington, DC, USA, 70–75. <https://doi.org/10.1109/CISIS.2015.35>
- [23] Johannes Wettinger, Uwe Breitenbücher, and Frank Leymann. 2014. Compensation-Based vs. Convergent Deployment Automation for Services Operated in the Cloud. In *Service-Oriented Computing - 12th International Conference, ICSOC 2014, Paris, France, November 3-6, 2014. Proceedings*. 336–350. https://doi.org/10.1007/978-3-662-45391-9_23
- [24] Jisoo Yang and Kang G. Shin. 2008. Using Hypervisor to Provide Data Secrecy for User Applications on a Per-page Basis. In *Proceedings of the Fourth ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE ’08)*. ACM, New York, NY, USA, 71–80. <https://doi.org/10.1145/1346256.1346267>
- [25] Fengzhe Zhang, Jin Chen, Haibo Chen, and Binyu Zang. 2011. CloudVisor: Retrofitting Protection of Virtual Machines in Multi-tenant Cloud with Nested Virtualization. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles (SOSP ’11)*. ACM, New York, NY, USA, 203–216. <https://doi.org/10.1145/2043556.2043576>